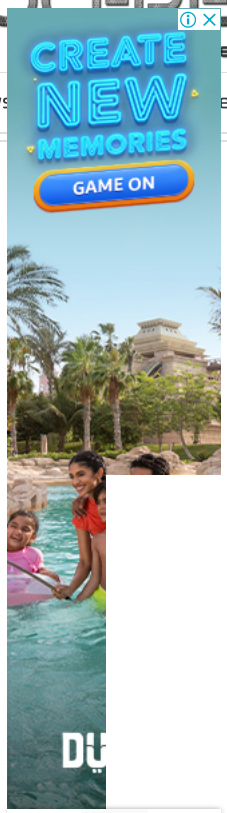
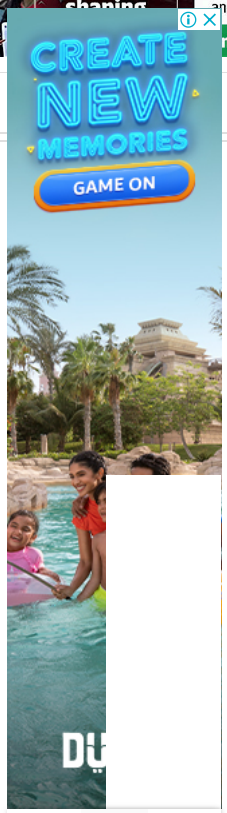
OCFREAKS! Technomaniacs
International Women's Day
I am leading change in my village,
Over 14 lakh representatives in
and Municipal Corp
Empowered Wo





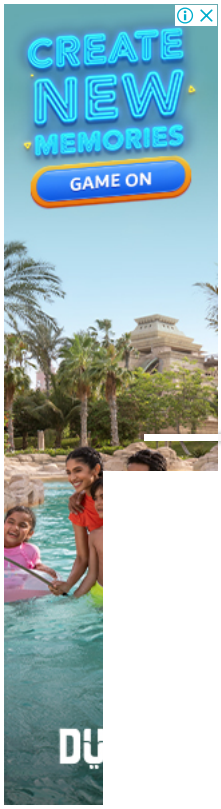
FOLLOW US

Search ...

SUBSCRIBE VIA EMAIL

Email Address

Subscribe



LPC1768 ADC Programming Tutorial

Posted By **Umang Gajera** Posted date: October 14, 2017 in: Embedded

Hi folks, in this tutorial we will go through ARM Cortex-M3 LPC1768 ADC programming tutorial. Basically we convert an Analog signal to its Digital version. We will also see LPC176x ADC Interfacing Example. This also applies to LPC1769 Microcontroller.

Table of Contents

ADC in ARM LPC176x
LPC176x ADC Registers
LPC1768 ADC Modes & Programming
LPC1768/LPC1769 ADC Example

ADC in ARM LPC176x

Analog to Digital Conversion is used when want to interface an external analog signal or when interfacing analog sensors, like for example a temperature sensor. The ADC block in ARM Cortex-M3 LPC1768 Microcontroller is based on Successive Approximation(SAR) conversion method. LPC1768 ADC Module uses 12-bit SAR having a conversion rate of 200 kHz. The Measurement range if from V_{REFN} to V_{REFP}, or commonly from 0V to ~3V. Maximum of 8 multiplexed inputs can be used for ADC.

Pins relating to ADC Module of LPC1768/LPC1769 :

Pin	Description
AD0.0 to AD0.7 (P0.23/24/25/26, P1.30/31, P0.3/2)	Analog input pins. **Note from Datasheet:** "If ADC is used, signal levels on analog input pins must not be above the level of V_{DDA} at any time. Otherwise, A/D converter readings will be invalid. If the A/D converter is not used in an application then the pins associated with A/D inputs can be used as 5V tolerant digital IO pins."
V_{REFP}, V_{REFN}	These reference voltage pins used for ADC and DAC.
V_{DDA}, V_{SSA}	V_{DDA} is Analog Power pin and V_{SSA} is Ground pin used to power the ADC module.

★ Analog Power Pin i.e V_{DDA} must be properly isolated/decoupled from V_{CC}, at bare minimum using a ferrite bead and a decoupling capacitor, to suppress noise present on V_{CC} rail and also to suppress MCU's switching noise which can be induced on the V_{CC} rail.

LPC176x ADC Registers

Lets see registers which are used for ADC programming.

1) ADCR – A/D Control Register : This is the main control register for AD0.

1. **Bits[7:0] – SEL:** Bit 'x'(in this group) is used to select pin A0.x in case of AD0.
2. **Bits[15:8] – CLKDIV:** ADC Peripheral clock i.e. PCLK_ADC0 is divided by CLKDIV+1 to get the ADC clock. Note that ADC clock speed must be <= 13Mhz! As per datasheet user must program the smallest value in this field which yields a clock speed of 4.5 MHz or a bit less.
3. **Bit[16] – BURST:** Set to 1 for doing repeated conversions, else 0 for software controlled conversions. Note: START bits must be set to 000 when BURST=1 or conversions will not start. Refer user manual for detailed info.
4. **Bit[21] – PDN :** Set it to 1 for powering up the ADC and making it operational. Set it to 0 for bringing it in powerdown mode.
5. **Bits[26:24] – START:** These bits are used to control the start of ADC conversion when BURST (bit 16) is set to 0. 000 = No start , 001 = Start the conversion now, for other values refer LPC17xx User Manual.
6. **Bit[27] – EDGE:** Set this bit to 1 to start the conversion on falling edge of the selected CAP/MAT signal and set this bit to 0 to start the conversion on rising edge of the selected signal. (Note: This bit is of used only in the case when the START contains a value between 010 to 111 as shown above.)
7. Other bits are reserved.

2) ADGDR – A/D Global Data Register : Contains the ADC's flags and the result of the most recent A/D conversion.

1. **Bits[15:4] – RESULT:** When DONE bit = 1, these bits give a binary fraction which represents the voltage on the pin AD0.x, in range V_{REFP} & V_{REFN}. Value of 0x0 indicates that voltage on the given pin was less than, equal to or greater than V_{REFN}. And a value of 0xFFF means that the input voltage was close to, equal to or greater than the V_{REFP}.
2. **Bits[26:24] – CHN:** Represents the channel from which RESULT bits were converted. 000= channel 0, 001= channel 1 and so on.
3. **Bit[30] – OVERRUN:** In burst mode this bit is 1 in case of an Overrun i.e. the result of previous conversion being lost(overwritten). This bit will be cleared after reading ADGDR.
4. **Bit[31] – DONE:** When ADC conversion completes this bit is 1. When this register(ADGDR) is read and ADCR is written, this bit gets cleared i.e. set to 0. If ADCR is written while a conversion is in progress then this bit is set and a new conversion is started.
5. Other bits are reserved.

4) ADDR0 to ADDR7 – A/D Data registers : This register contains the result of the most recent conversion completed on the corresponding channel [0 to 7]. Its structure is same as ADGDR except Bits[26:24] are not

FACEBOOK

Follow Pag

used/reserved.

5) ADSTAT – A/D Status register : This register contains DONE and OVERRUN flags for all of the A/D channels along with A/D interrupt flag.

1. **Bits[7:0] – DONE[7 to 0]:** Here xth bit mirrors DONEx status flag from the result register for A/D channel x.
2. **Bits[15:8] – OVERRUN[7 to 0]:** Even here the xth bit mirrors OVERRUNx status flag from the result register for A/D channel x
3. **Bit 16 – ADINT:** This bit represents the A/D interrupt flag. It is 1 when any of the individual A/D channel DONE flags is asserted and enabled to contribute to the A/D interrupt via the ADINTEN(given below) register.
4. Other bits are reserved.

LPC1768 ADC Modes, Setup and Programming

ADC modes in LPC1768 & LPC1769:

1. **Software controlled mode :** In Software mode only one conversion will be done at a time. To perform another conversion you will need to re-initiate the process. In software mode, only 1 bit in the SEL field of ADCR can be 1 i.e. only 1 Channel(i.e. Pin) can be selected for conversion at a time. Hence conversions can be done only any channel but one at a time.
2. **Hardware or Burst mode :** In Hardware or Burst mode, conversions are performed continuously on the selected channels in round-robin fashion. Since the conversions cannot be controlled by software, Overrun may occur in this mode. Overrun is the case when a previous conversion result is replaced by new conversion result without previous result being read i.e. the conversion is lost. Usually an interrupt is used in Burst mode to get the latest conversion results. This interrupt is triggered when conversion in one of the selected channel ends.

1A. Setting up and configuring ADC Module for software controlled mode

First, lets define some values which will help us in setting up ADCR register to configure & Initialize ADC block.

```
#define ADC_CLK_EN (1<<12)

#define SEL_AD0_0  (1<<0) //Select Channel AD0.0

#define CLKDIV     1 // ADC clock-divider (ADC_CLOCK=PCLK/CLKDIV+1)

#define PWRUP      (1<<21) //setting it to 0 will power it down

#define START_CNV  (1<<24) //001 for starting the conversion immediately

#define ADC_DONE   (1U<<31) //define it as unsigned value or compiler will throw #61-D warning

#define ADCR_SETUP_SCM ((CLKDIV<<8) | PowerUP)
//SCM = Software Controlled Mode
```

Now we assign ADCR_SETUP to ADCR along with channel selection information to select channels as required. Finally we assign(by ORing) START_NOW to ADCR to start the conversion process as shown:

```
LPC_SC->PCONP |= ADC_CLK_EN; //Enable ADC clock
LPC_ADC->ADCR =  ADCR_SETUP_SCM | SEL_AD0_0;
LPC_ADC->ADCR |= START_CNV;

//==OR==
LPC_SC->PCONP |= ADC_CLK_EN; //Enable ADC clock
LPC_ADC->ADCR =  ADCR_SETUP | SEL_AD0_0 | START_CNV;
```

1B. Fetching the conversion result in software controlled mode :

In software controlled mode we continuously monitor bit 31 in the corresponding channel data register ADDR. If bit 31 changes to 1 from 0, it means that current conversion has been completed and the result is ready. For example, if we are using channel 0 of AD0 then we monitor for changes in bit 31 as follows :

```
while((LPC_ADC->ADDR0 & ADC_DONE) == 0); //this loop will end when bit 31 of AD0DR6 changes to 1.
```

After this we extract the result which is stored in ADDR bits 4 to 15. Here we right shift ADDR by 6 places and force all other unrelated bits to 0 by using a 12-bit mask value of 0xFFF. 0xFFF is a mask containing 1's in bit locations 0 to 11 and rest 0's. In our case with ADDR0 being used, it can be done as follows :

```
result = (LPC_ADC->ADDR0>>4) & 0xFFF;
```

2A. Setting up and configuring ADC Module for Burst mode

Configuring ADC Module is similar to what was done in software controlled mode except here we use the CLKS bits and don't use the START bits in ADCR. ADC_DONE is also not applicable since we are using an ISR which gets triggered when a conversion completes on any of the enabled channels. Additionally, we define the following constants:

```
#define SEL_AD0_1 (0x2) //Select Channel AD0.1

#define BURST_ON  (1<<16) // 1 for on and 0 for off

#define ADCR_SETUP_BURST ((CLKDIV<<8) | BURST_ON | PWRUP)
```

We configure and setup the ADC module in a similar manner(as shown above) as follows :

```
LPC_SC->PCONP |= ADC_CLK_EN; //Enable ADC clock
LPC_ADC->ADCR =  ADCR_SETUP_BURST | SEL_AD0_0 | SEL_AD0_1;
```

Note that in this case we can select multiple channels for conversion when setting up AD0CR. START bits are not applicable here since the conversions starts as soon we setup AD0CR register.

2B. Fetching the conversion result in Burst mode :

In Burst mode we use an ISR which triggers at the completion of a conversion in any one of the channel. Now, we just need to find the Channel for which the conversion was done. For this we fetch the channel number from ADGDR which also stores the conversion result. Bits [26:24] in ADGDR contain the channel number. Hence, we shift it 24 places and use a 3-bit mask value of 0x7 as shown below:

```
unsigned long ADGDR_Read = LPC_ADC->ADGDR;
int channel = (ADGDR_Read>>24) & 0x7; //Extract Channel Number
```

After knowing the Channel number, we have 2 options to fetch the conversion result from. Either we can fetch it from ADGDR or from ADDRx of the corresponding channel. Lets use ADGDR for extracting the conversion result as follows:

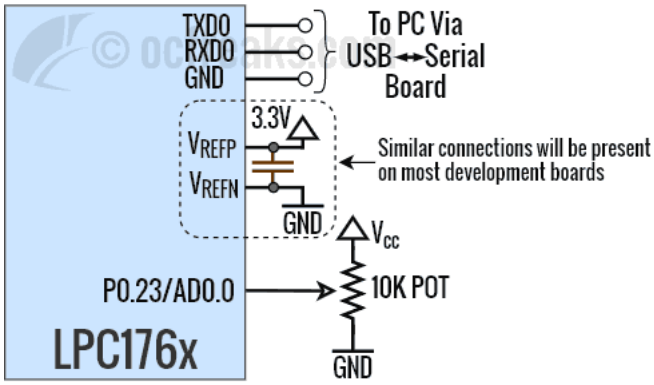
```
int currentResult = (ADGDR_Read>>4) & 0xFFF; //Extract Conversion Result
```

ARM Cortex-M3 LPC1768/LPC1769 ADC Example

Note that in this case no input protection nor filtering was required. But, when Interfacing external analog signals it is recommended to use some form of input protection.

Interfacing Potentiometer using ADC on LPC176x

This example performs Analog to Digital conversion in Software Controlled Mode. Here we use P0.23 as analog input for measuring the voltage. P0.23 corresponds to Channel 0 of AD0 i.e. AD0.0. For testing I had used a 10K potentiometer for ADC Interfacing and connected the middle leg to P0.23 of my LPC1768 development board. You can also use a 5K potentiometer for this ADC Example. Make sure your board V_{REFP}, V_{REFN}, V_{DDA} & V_{SSA} connections. These connections will be generally present on most LPC176x development boards. If not you can hook it up to V_{CC} and GND respectively using decoupling capacitors and ferrite beads to reduce noise. Schematic for this example is as shown below:



C/C++ Source Code :

```
    LPC_SC->PCONP |= ADC_CLK_EN; //Enable ADC clock
    LPC_ADC->ADCR =  ADCR_SETUP_SCM | SEL_AD0_0;
    LPC_PINCON->PINSEL1 |= (1<<14) ; //select AD0.0 for P0.23
    int result = 0;
    float volts = 0;

    printf("OCFreaks.com LPC176x ADC Tutorial Example 1.\nSoftware Controlled ADC Mode on AD0.0 Channel

    while(1)
    {
        LPC_ADC->ADCR |= START_CNV; //Start new Conversion

        while((LPC_ADC->ADDR0 & ADC_DONE) == 0); //Wait untill conversion is finished

        result = (LPC_ADC->ADDR0>>4) & 0xFFF; //12 bit Mask to extract result

        volts = (result*VREF)/4096.0; //Convert result to Voltage

        printf("AD0.0 = %dmV\n" , (int)(volts*1000)); //Display milli-volts

        delayMS(500); //Slowing down Updates to 2 Updates per second
    }

    //return 0; //This won't execute
}
```

Serial Output :

```
COM7 - PuTTYtel
OCFreaks.com LPC176x ADC Tutorial Example 1.
Software Controlled ADC Mode on AD0.0 Channel.
AD0.0 = 17mV
AD0.0 = 19mV
AD0.0 = 555mV
AD0.0 = 780mV
AD0.0 = 928mV
AD0.0 = 1044mV
AD0.0 = 1259mV
AD0.0 = 1434mV
AD0.0 = 1596mV
AD0.0 = 2281mV
AD0.0 = 2633mV
AD0.0 = 2925mV
AD0.0 = 3264mV
AD0.0 = 3299mV
AD0.0 = 3299mV
AD0.0 = 3299mV
```

`ocf_lpc176x_lib.h` header used in the source also contains code for **retargeting printf()**. Check how to retarget printf() using UART tutorial for more.

KEIL ARM uV5 Project for example given above on GitHub @ LPC176x ADC Interfacing Example 1 [Successfully tested on Keil uV5.23], Download Project Zip. You can find the HEX file inside objects folder.

Last updated on November 18, 2017

ST's Smart Power Switches

Like Tweet Share Share Share

« Previous
LPC2148 DAC Programming Tutorial

Next »
Western Digital unveils new Innovation in MAMR Technology, 40 TB HDDs by 2025

ABOUT THE AUTHOR

Umang Gajera

Earn Upto 5k Daily By This Method of
TradeWise Learn More

ALSO ON OCFREAKS!

8 years ago · 1 comment
QuantumZERO QZ-HD01 USB3.0 SATA Hard ...

7 years ago · 1 comment
LPC1114 PWM Programming Tutorial

Sponsored

Earn Upto 5k Daily By This Method of Intraday Trading !

TradeWise

Learn More

Renewed Ayurvedic Medicine for Healthy Blood Sugar Support

Goodveda's Insulux

Shop Now

Last 36 residences before prices rise

Godrej Reserve Kandivali East

Book Now

Neurologist: 97% of People With Neuropathy Don't Know This Crucial Thing

Nerve Relief

These Are The Most Beautiful Women In The World

5minstory.com

Crossout: New Apocalyptic MMO

Crossout

Play Now

4 Comments Login ▾

G Join the discussion…

LOG IN WITH OR SIGN UP WITH DISQUS ?

Name

Share Best Newest Oldest

B **Bhim Shankar**
7 years ago

Could you please give the code for burst mode also. After getting the channel number by using "int channel = (ADGDR_Read>>24) & 0x7; //Extract Channel Number" how we can get the particular channel ADC value.

0 0 Reply

Ava **Irwan**
7 years ago edited

Hi admin,

I am sorry, I am new in ARM
could you explain to me what means of (->) in LPC_SC->PCONP |= ADC_CLK_EN;
could you give me guide to start learning ARM?

Thanks
Putra

0 0 Reply

Power_user_EX Mod → Irwan
7 years ago edited

Hi Putra, "->" is an operator used to deference pointers. In our case we are deferencing member "PCONP" of pointer to a structure named "LPC_SC".

I have mentioned in http://www.ocfreaks.com/lpc... under GPIO register section.

For more refer : https://en.wikipedia.org/wi...

-Regards

6 0 Reply

Ava **Irwan** → Power_user_EX
7 years ago

Thank you for your reply, now I am a little understand
I want to know some syntax in LPC17xx.h
I am not detail understand c programming,
could you explain to me what means of this
#define LPC_SC ((LPC_SC_TypeDef *) LPC_SC_BASE)

LPC_SC_BASE is 0x400FC000
this is to defince LPC_SC address, right?
how to translate this code,

do you have a embedded c program language book or document to understand it?
thanks

0 0 Reply

Subscribe Privacy Do Not Sell My Data

Sponsored

Renewed Ayurvedic Medicine for Healthy Blood Sugar Support

Goodveda's Insulux

Earn Upto 5k Daily By This Method of Intraday Trading !

TradeWise

Learn More

Neurologist: 97% of People With Neuropathy Don't Know This Crucial Thing

Nerve Relief

These Are The Most Beautiful Women In The World

5minstory.com

Top 10 Solana Ecosystem Tokens To Invest In 2025

ZebPay

Learn More

Crossout: New Apocalyptic MMO

Crossout

Play Now

(C) OCFreaks! 2011-24.